

Contoso Retail — Azure SRE Agent
End-to-End Demo Report

ODE Reduction Demo Kit — from customer impact to evidence-backed recovery

Generated: 2026-08-06T10:57:23.136Z
Repository: pronmalw/azure-sre-agent-ode-demo-kit (dev branch)
Run mode: Local dev (in-memory fallback, API :3001 / Web :3000)

1. Executive Summary

This report documents a full end-to-end run of the Contoso Retail SRE Agent ODE Reduction demo. All 14 chaos scenario fixtures were exercised against the live classifier engine (`src/api/src/services/sre-classifier.service.ts`), and **14/14 classifications matched the fixture's expected outcome**. In addition, a real end-to-end incident was triggered live (not from a canned fixture): chaos toggles were enabled, synthetic customer traffic was generated, the SRE Agent investigated the live telemetry, a safe recovery action was executed, and recovery was verified against 9 objective criteria.

The purpose of this demo is to show how Azure SRE Agent reduces **Operational Delivery Effort (ODE)** by moving from raw customer impact signals to an evidence-backed, ownership-classified, safe action plan — without unnecessary escalation to Azure support when the issue is application-side.

2. How the SRE Agent Investigates (Prompt & Method)

In a real deployment, an engineer opens sre.azure.com → contoso-sre-agent and issues the master investigation prompt below. The agent runs a full diagnostic sweep across Application Insights, Log Analytics, Cosmos DB, and Azure SQL telemetry for every known failure scenario in parallel, classifies each active finding, and produces a structured report before requesting approval to remediate.

Investigation logic mirrored in this demo (in-memory classifier):

- If Cosmos 429 count > 10 AND server-side latency < 100ms AND a client-pattern chaos toggle is active → APPLICATION_OR_CLIENT_SIDE (HIGH confidence) — the SDK/query pattern is the cause, Azure is healthy.
- If server-side latency > 500ms AND 429s > 10 AND no toggles active → AZURE_SERVICE_SIDE (MEDIUM) — possible genuine platform degradation.
- If host CPU > 70% AND p99 > 1000ms AND only the CPU-burn toggle is active → HOST_OR_RUNTIME (HIGH).
- If only data-model toggles (cross-partition query / missing index / point-read misuse) are active → DATABASE_CONFIG_OR_DATA_MODEL (HIGH).
- If a SQL chaos toggle is active AND SQL latency > 1000ms → SQL_DATABASE_OR_SCHEMA (HIGH).
- Otherwise → INSUFFICIENT_EVIDENCE (LOW) — no action taken, avoiding false-positive escalation.

For every finding the agent also states **ruled-out causes**, **missing evidence**, an **executive summary**, an **engineering RCA**, **immediate safe actions** (no approval needed), **approval-required actions**, explicit **DO NOT DO** guidance, and an **escalation recommendation** — then it waits for human approval before remediating (Phase 4/5 of the master prompt).

Master investigation prompt (paste into sre.azure.com)

```
ROLE: You are the Azure SRE Agent for the Contoso Retail platform. Your goal is to triage ALL active incidents, identify every root cause, produce a structured incident report (formatted for PDF export), and then – only with explicit user approval – execute the safest remediation actions.
ENVIRONMENT Application : Contoso Retail (e-commerce) Resource Group: rg-sreagent-ode-demo (West Europe) AKS Cluster : aks-contoso-sreagent-demo, namespace: contoso-retail App Insights : appi-euqicty6gdfys (InstrumentationKey 91e55584-e782-42a0-8567-3ccdc3a3288f) Log Analytics : law-euqicty6gdfys (WorkspaceId 527754b0-cdd5-467a-9128-4f3a4a173b16) Cosmos DB : cosmos-euqicty6gdfys.documents.azure.com:443 Azure SQL : sql-euqicty6gdfys.database.windows.net / contoso-retail-db Time Window : LAST 60 MINUTES ONLY – from now() - 1h to now()
PHASE 1 – FULL DIAGNOSTIC SWEEP (run all checks in parallel)
Run EVERY check below. Do not skip any section even if earlier sections look healthy.
SCENARIO 1 CHECK: Deployment Gone Wrong
S1-A. Query App Insights for p99 end-to-end latency in the last 60 min: requests | where timestamp > ago(1h) | summarize p99=percentile(duration,99) by bin(timestamp,5m) S1-B. Count Cosmos DB 429 (TooManyRequests) errors: traces | where timestamp > ago(1h) and message has "429" | summarize count() by bin(timestamp,5m) S1-C. Check host CPU on AKS pods (from Container Insights): Perf | where TimeGenerated > ago(1h) and ObjectName == "K8SContainer" and CounterName == "cpuUsageNanoCores" | summarize avg(CounterValue)/1000000 by bin(TimeGenerated,5m), InstanceName S1-D. Look for multiple CosmosClient instantiation events: traces | where timestamp > ago(1h) and message has "CosmosClient created" S1-E. Detect hot-partition symptoms (single partition key receiving >80% of RU): traces | where timestamp > ago(1h) and message has "hot_partition"
SCENARIO 2 CHECK: Database Architect's Mistake
S2-A. Measure Cosmos DB RU consumption trend vs baseline: customMetrics | where timestamp > ago(1h) and name == "cosmos_ru_consumed" | summarize avg(value), max(value) by bin(timestamp,5m) S2-B. Detect cross-partition fan-out queries (high RU, low result count): customEvents | where timestamp > ago(1h) and name == "cosmos_query" | where customDimensions.ruConsumed > 100 S2-C. Detect large document writes (>100KB): traces | where timestamp > ago(1h) and message has "large_document" S2-D. Detect missing index warnings: traces | where timestamp > ago(1h) and message has "missing_index" or message has "IndexingPolicy" S2-E. Check point-read misuse (querying full partition for a single ID): customEvents | where timestamp > ago(1h) and name == "cosmos_query" | where customDimensions.queryType == "full_scan" and customDimensions.resultCount == 1
SCENARIO 3 CHECK: Resource Exhaustion
S3-A. Pod CPU utilisation over the hour (flag any pod > 80%): ContainerInventory | join kind=inner ( Perf | where TimeGenerated > ago(1h) and ObjectName == "K8SContainer" and CounterName == "cpuUsageNanoCores" ) on InstanceName | summarize
```

Quick per-scenario prompts used in this run

```
# Ready-to-use SRE Agent prompts 1. "Investigate current Contoso Retail latency regression and classify the likely owner." 2.
"Summarize Cosmos DB client-side vs server-side latency evidence for the last 30 minutes." 3. "Explain whether Azure SQL is
implicated in the current checkout failure." 4. "List safe actions we can take immediately without customer approval." 5.
```

"Generate a Teams update for the current incident using the cached report." 6. "Verify whether recovery criteria are satisfied after we reset the chaos toggles." 7. "Compare the current incident to the hot-partition scenario fixture and highlight similarities."

3. Scenario Coverage Summary (14 scenarios)

Scenario	Description	Actual classification	Confidence	Result	Primary owner
Azure service-side issue	No chaos toggles, but service-side latency spikes.	AZURE_SERVICE_SIDE	MEDIUM	MATCH	Azure platform team
Contoso mixed incident	All chaos toggles enabled to create a blended client, data model, host, and SQL issue.	APPLICATION_OR_CLIENT_SIDE	HIGH	MATCH	Contoso Retail application engineering
Cross partition query	Cross partition fan-out query active.	DATABASE_CONFIG_OR_DATA_MODEL	HIGH	MATCH	Contoso Retail data platform team
Healthy baseline	All chaos toggles off and healthy application metrics.	INSUFFICIENT_EVIDENCE	LOW	MATCH	Application on-call
High CPU host saturation	Only CPU burn path enabled.	HOST_OR_RUNTIME	HIGH	MATCH	AKS platform and runtime team
Hot partition 429	Only hot partition enabled.	APPLICATION_OR_CLIENT_SIDE	HIGH	MATCH	Contoso Retail application engineering
Insufficient evidence	Healthy metrics without a strong signal.	INSUFFICIENT_EVIDENCE	LOW	MATCH	Application on-call
Large document write	Oversized documents on write path.	APPLICATION_OR_CLIENT_SIDE	HIGH	MATCH	Contoso Retail application engineering

Metadata throttling	Only metadata reads enabled.	APPLICATION_OR_CLIENT_SIDEHIGH	MATCH	Contoso Retail application engineering
Missing indexing policy	Reads routed to unindexed container.	DATABASE_CONFIG_OR_DATA_MODELHIGH	MATCH	Contoso Retail data platform team
Multiple Cosmos clients	SDK client created per request.	APPLICATION_OR_CLIENT_SIDEHIGH	MATCH	Contoso Retail application engineering
Point read misuse	Query used where point read should be used.	DATABASE_CONFIG_OR_DATA_MODELHIGH	MATCH	Contoso Retail data platform team
SQL connection pool pressure	Connections are held too long.	SQL_DATABASE_OR_SCHEMAHIGH	MATCH	Contoso Retail SQL/data engineering team
SQL slow query	Artificial SQL slowness injected.	SQL_DATABASE_OR_SCHEMAHIGH	MATCH	Contoso Retail SQL/data engineering team

4. Detailed Findings per Scenario

Azure service-side issue (azure-service-side-issue)

No chaos toggles, but service-side latency spikes.

Primary classification	AZURE_SERVICE_SIDE MATCH
Secondary classification(s)	—
Confidence	MEDIUM
Primary owner	Azure platform team
Customer impact	Customers are experiencing degraded catalogue and checkout performance (p99 2800ms, checkout success 93%).

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 2800ms). Investigation evidence indicates the root cause is service-side, specifically: Client-side p99 latency: 2800ms vs server-side Cosmos latency: 650ms — 2150ms gap indicates client/SDK or host overhead; Cosmos DB 429 RateLimitExceeded: 50 in last 5 min on products container. Azure Cosmos DB service-side health remained degraded (server-side latency: 650ms). Azure SQL Database remained healthy. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- Client-side p99 latency: 2800ms vs server-side Cosmos latency: 650ms — 2150ms gap indicates client/SDK or host overhead
- Cosmos DB 429 RateLimitExceeded: 50 in last 5 min on products container

Ruled out:

- Azure SQL Database issue ruled out: SQL query latency 80ms, error count 0
- Host resource exhaustion ruled out: CPU at 22%
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review
- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review
- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT blame Azure SQL when SQL signals are healthy — SQL latency is 80ms
- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Consider escalating to Azure Support — server-side signals suggest possible service degradation. Include CRI/subscription ID and telemetry exports.

Root cause / permanent fix: Capture service-side evidence and coordinate with Azure Support while maintaining customer communications.

Engineering RCA: Telemetry shows p99 latency at 2800ms with 50 Cosmos 429 events, 450 RU/min consumption, host CPU 22%, and SQL latency 80ms. The active toggles (none) directly align with the observed failure signature. The client-to-server latency delta of 2150ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

Contoso mixed incident (contoso-sql-cosmos-mixed-incident)

All chaos toggles enabled to create a blended client, data model, host, and SQL issue.

Primary classification	APPLICATION_OR_CLIENT_SIDE MATCH
Secondary classification(s)	RECENT_CHANGE_OR_DEPLOYMENT, HOST_OR_RUNTIME, DATABASE_CONFIG_OR_DATA_MODEL, SQL_DATABASE_OR_SCHEMA
Confidence	HIGH
Primary owner	Contoso Retail application engineering
Customer impact	Customers are experiencing degraded catalogue and checkout performance (p99 3200ms, checkout success 62%).

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 3200ms). Investigation evidence indicates the root cause is application-side, specifically: Client-side p99 latency: 3200ms vs server-side Cosmos latency: 20ms — 3180ms gap indicates client/SDK or host overhead; Cosmos DB 429 RateLimitExceeded: 55 in last 5 min on products container; Hot partition pattern active: all writes routing to fixed partition key 'hot-category'. Azure Cosmos DB service-side health remained nominal (server-side latency: 20ms). Azure SQL requires further validation. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- Client-side p99 latency: 3200ms vs server-side Cosmos latency: 20ms — 3180ms gap indicates client/SDK or host overhead
- Cosmos DB 429 RateLimitExceeded: 55 in last 5 min on products container
- Hot partition pattern active: all writes routing to fixed partition key 'hot-category'
- Multiple CosmosClient instances: new client created per request instead of singleton
- Per-request metadata reads: container properties re-fetched on every operation
- Cross-partition fan-out query: SELECT * FROM c without partition key filter
- Large document writes: ~52KB documents (recommended <1KB for typical catalogue items)
- Unindexed query on ProductsNoIndex: full scan, high RU expected

- Query used instead of ReadItem: `SELECT * FROM c WHERE c.id=@id` vs `ReadItemAsync(id,pk)`
- Host CPU: 85% — CPU burn path active (fibonacci loop on request path)
- SQL query latency: 3200ms — artificial slow query path enabled
- SQL connection pool pressure: connections held beyond normal release
- Recent deployment/change event detected — correlates with incident start
- High RU consumption: 1600 RU/min — 16x above healthy baseline
- Azure Cosmos DB server-side latency: 20ms — within normal range, indicating service is healthy

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 20ms is healthy

Immediate safe actions (no approval needed):

- Disable hot partition chaos toggle — restore natural partition key routing by `categoryId/userId`
- Stop creating new `CosmosClient` per request — use singleton initialized at startup
- Remove per-request metadata reads — cache container properties at startup
- Disable CPU burn path — remove synchronous Fibonacci computation from request path
- Replace cross-partition query with targeted query using `WHERE c.categoryId = @categoryId`
- Restore indexing policy on `ProductsNoIndex`, or route queries back to indexed `Products` container
- Replace query with `ReadItemAsync(id, partitionKey)` for known `id+pk` combinations
- Reduce document size — extract large blobs to Azure Blob Storage, keep documents <1KB
- Disable SQL slow query simulation — verify no N+1 query patterns in checkout path
- Disable SQL connection pressure — ensure connections returned to pool promptly
- Verify singleton `CosmosClient` initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on `ProductsNoIndex` container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review
- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review

- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT treat this as Azure/Cosmos DB service outage — server-side metrics are healthy
- DO NOT blindly increase provisioned RU/s — this masks the hot partition root cause
- DO NOT ignore hot partition evidence — increasing throughput without fixing key distribution does not resolve root cause
- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Do NOT escalate to Azure/Cosmos support. Evidence strongly indicates application-side patterns. Escalate only if server-side latency exceeds 200ms p99, availability drops below 99.9%, or Azure Service Health shows regional impact.

Root cause / permanent fix: Refactor Cosmos access patterns to use a singleton client, targeted point reads, well-distributed partition keys, and lean document shapes.

Engineering RCA: Telemetry shows p99 latency at 3200ms with 55 Cosmos 429 events, 1600 RU/min consumption, host CPU 85%, and SQL latency 3200ms. The active toggles (hotPartition, metadataThrottling, multipleClients, highCpu, crossPartitionQuery, largeDocument, missingIndexing, pointReadMisuse, sqlSlowQuery, sqlConnectionPressure) directly align with the observed failure signature. The client-to-server latency delta of 3180ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

Cross partition query (cross-partition-query)

Cross partition fan-out query active.

Primary classification	DATABASE_CONFIG_OR_DATA_MODEL MATCH
Secondary classification(s)	—
Confidence	HIGH
Primary owner	Contoso Retail data platform team
Customer impact	Low-severity symptoms observed with p99 latency 980ms and checkout success 96%.

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 980ms). Investigation evidence indicates the root cause is most likely associated with the active failure domain, specifically: Client-side p99 latency: 980ms vs server-side Cosmos latency: 18ms — 962ms gap indicates client/SDK or host overhead; Cosmos DB 429 RateLimitExceeded: 8 in last 5 min on products container; Cross-partition fan-out query: SELECT * FROM c without partition key filter. Azure Cosmos DB service-side health remained nominal (server-side latency: 18ms). Azure SQL Database remained healthy. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- Client-side p99 latency: 980ms vs server-side Cosmos latency: 18ms — 962ms gap indicates client/SDK or host overhead
- Cosmos DB 429 RateLimitExceeded: 8 in last 5 min on products container
- Cross-partition fan-out query: SELECT * FROM c without partition key filter
- High RU consumption: 720 RU/min — 7.2x above healthy baseline

- Azure Cosmos DB server-side latency: 18ms — within normal range, indicating service is healthy

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 18ms is healthy
- Azure SQL Database issue ruled out: SQL query latency 48ms, error count 0
- Host resource exhaustion ruled out: CPU at 25%
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Replace cross-partition query with targeted query using WHERE c.categoryId = @categoryId
- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review
- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review
- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT blame Azure SQL when SQL signals are healthy — SQL latency is 48ms
- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Keep escalation within the Contoso engineering response bridge while corrective actions are validated. Escalate externally only if ruled-out causes become active signals.

Root cause / permanent fix: Correct the Cosmos indexing and query design, then validate RU efficiency with representative load.

Engineering RCA: Telemetry shows p99 latency at 980ms with 8 Cosmos 429 events, 720 RU/min consumption, host CPU 25%, and SQL latency 48ms. The active toggles (crossPartitionQuery) directly align with the observed failure signature. The client-to-server latency delta of 962ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

Healthy baseline (healthy-baseline)

All chaos toggles off and healthy application metrics.

Primary classification	INSUFFICIENT_EVIDENCE MATCH
Secondary classification(s)	—
Confidence	LOW
Primary owner	Application on-call
Customer impact	Low-severity symptoms observed with p99 latency 95ms and checkout success 100%.

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 95ms). Investigation evidence indicates the root cause is most likely associated with the active failure domain, specifically: No strong anomaly detected across client, server, host, or SQL telemetry during the sampled window.. Azure Cosmos DB service-side health remained nominal (server-side latency: 12ms). Azure SQL Database remained healthy. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- No strong anomaly detected across client, server, host, or SQL telemetry during the sampled window.

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 12ms is healthy
- Azure SQL Database issue ruled out: SQL query latency 32ms, error count 0
- Host resource exhaustion ruled out: CPU at 18%
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review
- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review
- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT blame Azure SQL when SQL signals are healthy — SQL latency is 32ms
- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Keep escalation within the Contoso engineering response bridge while corrective actions are validated. Escalate externally only if ruled-out causes become active signals.

Root cause / permanent fix: Collect additional telemetry to determine a durable corrective action.

Engineering RCA: Telemetry shows p99 latency at 95ms with 0 Cosmos 429 events, 40 RU/min consumption, host CPU 18%, and SQL latency 32ms. The active toggles (none) directly align with the observed failure signature. The client-to-server latency delta of 83ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

High CPU host saturation (high-cpu-host-saturation)

Only CPU burn path enabled.

Primary classification	HOST_OR_RUNTIME MATCH
Secondary classification(s)	—
Confidence	HIGH
Primary owner	AKS platform and runtime team
Customer impact	Customers are experiencing degraded catalogue and checkout performance (p99 2400ms, checkout success 89%).

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 2400ms). Investigation evidence indicates the root cause is most likely associated with the active failure domain, specifically: Client-side p99 latency: 2400ms vs server-side Cosmos latency: 12ms — 2388ms gap indicates client/SDK or host overhead; Host CPU: 85% — CPU burn path active (fibonacci loop on request path); Azure Cosmos DB server-side latency: 12ms — within normal range, indicating service is healthy. Azure Cosmos DB service-side health remained nominal (server-side latency: 12ms). Azure SQL Database remained healthy. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- Client-side p99 latency: 2400ms vs server-side Cosmos latency: 12ms — 2388ms gap indicates client/SDK or host overhead
- Host CPU: 85% — CPU burn path active (fibonacci loop on request path)
- Azure Cosmos DB server-side latency: 12ms — within normal range, indicating service is healthy

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 12ms is healthy
- Azure SQL Database issue ruled out: SQL query latency 65ms, error count 0
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Disable CPU burn path — remove synchronous Fibonacci computation from request path
- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review
- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review
- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT blame Azure SQL when SQL signals are healthy — SQL latency is 65ms
- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Keep escalation within the Contoso engineering response bridge while corrective actions are validated. Escalate externally only if ruled-out causes become active signals.

Root cause / permanent fix: Remove synchronous CPU-heavy work from the request path and right-size AKS runtime capacity.

Engineering RCA: Telemetry shows p99 latency at 2400ms with 0 Cosmos 429 events, 75 RU/min consumption, host CPU 85%, and SQL latency 65ms. The active toggles (highCpu) directly align with the observed failure signature. The client-to-server latency delta of 2388ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

Hot partition 429 (hot-partition-429)

Only hot partition enabled.

Primary classification	APPLICATION_OR_CLIENT_SIDE MATCH
Secondary classification(s)	—
Confidence	HIGH
Primary owner	Contoso Retail application engineering
Customer impact	Customers are experiencing degraded catalogue and checkout performance (p99 1400ms, checkout success 91%).

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 1400ms). Investigation evidence indicates the root cause is application-side, specifically: Client-side p99 latency: 1400ms vs server-side Cosmos latency: 18ms — 1382ms gap indicates client/SDK or host overhead; Cosmos DB 429 RateLimitExceeded: 25 in last 5 min on products container; Hot partition pattern active: all writes routing to fixed partition key 'hot-category'. Azure Cosmos DB service-side health remained nominal (server-side latency: 18ms). Azure SQL Database remained healthy. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- Client-side p99 latency: 1400ms vs server-side Cosmos latency: 18ms — 1382ms gap indicates client/SDK or host overhead
- Cosmos DB 429 RateLimitExceeded: 25 in last 5 min on products container

- Hot partition pattern active: all writes routing to fixed partition key 'hot-category'
- High RU consumption: 780 RU/min — 7.8x above healthy baseline
- Azure Cosmos DB server-side latency: 18ms — within normal range, indicating service is healthy

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 18ms is healthy
- Azure SQL Database issue ruled out: SQL query latency 45ms, error count 0
- Host resource exhaustion ruled out: CPU at 22%
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Disable hot partition chaos toggle — restore natural partition key routing by categoryId/userId
- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review
- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review
- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT treat this as Azure/Cosmos DB service outage — server-side metrics are healthy
- DO NOT blindly increase provisioned RU/s — this masks the hot partition root cause
- DO NOT ignore hot partition evidence — increasing throughput without fixing key distribution does not resolve root cause
- DO NOT blame Azure SQL when SQL signals are healthy — SQL latency is 45ms
- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Do NOT escalate to Azure/Cosmos support. Evidence strongly indicates application-side patterns. Escalate only if server-side latency exceeds 200ms p99, availability drops below 99.9%, or Azure Service Health shows regional impact.

Root cause / permanent fix: Refactor Cosmos access patterns to use a singleton client, targeted point reads, well-distributed partition keys, and lean document shapes.

Engineering RCA: Telemetry shows p99 latency at 1400ms with 25 Cosmos 429 events, 780 RU/min consumption, host CPU 22%, and SQL latency 45ms. The active toggles (hotPartition) directly align with the observed failure signature. The client-to-server latency delta of 1382ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

Insufficient evidence (insufficient-evidence)

Healthy metrics without a strong signal.

Primary classification	INSUFFICIENT_EVIDENCE MATCH
Secondary classification(s)	—
Confidence	LOW
Primary owner	Application on-call
Customer impact	Low-severity symptoms observed with p99 latency 120ms and checkout success 100%.

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 120ms). Investigation evidence indicates the root cause is most likely associated with the active failure domain, specifically: No strong anomaly detected across client, server, host, or SQL telemetry during the sampled window.. Azure Cosmos DB service-side health remained nominal (server-side latency: 11ms). Azure SQL Database remained healthy. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- No strong anomaly detected across client, server, host, or SQL telemetry during the sampled window.

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 11ms is healthy
- Azure SQL Database issue ruled out: SQL query latency 28ms, error count 0
- Host resource exhaustion ruled out: CPU at 19%
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review

- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review
- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT blame Azure SQL when SQL signals are healthy — SQL latency is 28ms
- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Keep escalation within the Contoso engineering response bridge while corrective actions are validated. Escalate externally only if ruled-out causes become active signals.

Root cause / permanent fix: Collect additional telemetry to determine a durable corrective action.

Engineering RCA: Telemetry shows p99 latency at 120ms with 0 Cosmos 429 events, 35 RU/min consumption, host CPU 19%, and SQL latency 28ms. The active toggles (none) directly align with the observed failure signature. The client-to-server latency delta of 109ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

Large document write (large-document-write)

Oversized documents on write path.

Primary classification	APPLICATION_OR_CLIENT_SIDE MATCH
Secondary classification(s)	—
Confidence	HIGH
Primary owner	Contoso Retail application engineering
Customer impact	Customers are experiencing degraded catalogue and checkout performance (p99 1100ms, checkout success 94%).

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 1100ms). Investigation evidence indicates the root cause is application-side, specifically: Client-side p99 latency: 1100ms vs server-side Cosmos latency: 16ms — 1084ms gap indicates client/SDK or host overhead; Cosmos DB 429 RateLimitExceeded: 12 in last 5 min on products container; Large document writes: ~52KB documents (recommended <1KB for typical catalogue items). Azure Cosmos DB service-side health remained nominal (server-side latency: 16ms). Azure SQL Database remained healthy. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- Client-side p99 latency: 1100ms vs server-side Cosmos latency: 16ms — 1084ms gap indicates client/SDK or host overhead
- Cosmos DB 429 RateLimitExceeded: 12 in last 5 min on products container
- Large document writes: ~52KB documents (recommended <1KB for typical catalogue items)
- High RU consumption: 900 RU/min — 9x above healthy baseline
- Azure Cosmos DB server-side latency: 16ms — within normal range, indicating service is healthy

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 16ms is healthy
- Azure SQL Database issue ruled out: SQL query latency 58ms, error count 0
- Host resource exhaustion ruled out: CPU at 28%
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Reduce document size — extract large blobs to Azure Blob Storage, keep documents <1KB
- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review
- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review
- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT treat this as Azure/Cosmos DB service outage — server-side metrics are healthy
- DO NOT blindly increase provisioned RU/s — this masks the hot partition root cause
- DO NOT ignore hot partition evidence — increasing throughput without fixing key distribution does not resolve root cause
- DO NOT blame Azure SQL when SQL signals are healthy — SQL latency is 58ms
- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Do NOT escalate to Azure/Cosmos support. Evidence strongly indicates application-side patterns. Escalate only if server-side latency exceeds 200ms p99, availability drops below 99.9%, or Azure Service Health shows regional impact.

Root cause / permanent fix: Refactor Cosmos access patterns to use a singleton client, targeted point reads, well-distributed partition keys, and lean document shapes.

Engineering RCA: Telemetry shows p99 latency at 1100ms with 12 Cosmos 429 events, 900 RU/min consumption, host CPU 28%, and SQL latency 58ms. The active toggles (largeDocument) directly align with the observed failure signature. The client-to-server latency delta of 1084ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

Metadata throttling (metadata-throttling)

Only metadata reads enabled.

Primary classification	APPLICATION_OR_CLIENT_SIDE MATCH
Secondary classification(s)	—
Confidence	HIGH
Primary owner	Contoso Retail application engineering
Customer impact	Customers are experiencing degraded catalogue and checkout performance (p99 1200ms, checkout success 95%).

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 1200ms). Investigation evidence indicates the root cause is application-side, specifically: Client-side p99 latency: 1200ms vs server-side Cosmos latency: 15ms — 1185ms gap indicates client/SDK or host overhead; Cosmos DB 429 RateLimitExceeded: 15 in last 5 min on products container; Per-request metadata reads: container properties re-fetched on every operation. Azure Cosmos DB service-side health remained nominal (server-side latency: 15ms). Azure SQL Database remained healthy. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- Client-side p99 latency: 1200ms vs server-side Cosmos latency: 15ms — 1185ms gap indicates client/SDK or host overhead
- Cosmos DB 429 RateLimitExceeded: 15 in last 5 min on products container
- Per-request metadata reads: container properties re-fetched on every operation
- High RU consumption: 650 RU/min — 6.5x above healthy baseline
- Azure Cosmos DB server-side latency: 15ms — within normal range, indicating service is healthy

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 15ms is healthy
- Azure SQL Database issue ruled out: SQL query latency 50ms, error count 0
- Host resource exhaustion ruled out: CPU at 24%
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Remove per-request metadata reads — cache container properties at startup
- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review

- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review
- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT treat this as Azure/Cosmos DB service outage — server-side metrics are healthy
- DO NOT blindly increase provisioned RU/s — this masks the hot partition root cause
- DO NOT ignore hot partition evidence — increasing throughput without fixing key distribution does not resolve root cause
- DO NOT blame Azure SQL when SQL signals are healthy — SQL latency is 50ms
- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Do NOT escalate to Azure/Cosmos support. Evidence strongly indicates application-side patterns. Escalate only if server-side latency exceeds 200ms p99, availability drops below 99.9%, or Azure Service Health shows regional impact.

Root cause / permanent fix: Refactor Cosmos access patterns to use a singleton client, targeted point reads, well-distributed partition keys, and lean document shapes.

Engineering RCA: Telemetry shows p99 latency at 1200ms with 15 Cosmos 429 events, 650 RU/min consumption, host CPU 24%, and SQL latency 50ms. The active toggles (metadataThrottling) directly align with the observed failure signature. The client-to-server latency delta of 1185ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

Missing indexing policy (missing-indexing-policy)

Reads routed to unindexed container.

Primary classification	DATABASE_CONFIG_OR_DATA_MODEL MATCH
Secondary classification(s)	—
Confidence	HIGH
Primary owner	Contoso Retail data platform team
Customer impact	Customers are experiencing degraded catalogue and checkout performance (p99 1250ms, checkout success 97%).

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 1250ms). Investigation evidence indicates the root cause is most likely associated with the active failure domain, specifically: Client-side p99 latency: 1250ms vs server-side Cosmos latency: 18ms — 1232ms gap indicates client/SDK or host overhead; Cosmos DB 429 RateLimitExceeded: 9 in last 5 min on products container; Unindexed query on ProductsNoIndex: full scan, high RU expected. Azure Cosmos DB service-side health remained nominal (server-side latency: 18ms). Azure SQL Database remained healthy. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- Client-side p99 latency: 1250ms vs server-side Cosmos latency: 18ms — 1232ms gap indicates client/SDK or host overhead
- Cosmos DB 429 RateLimitExceeded: 9 in last 5 min on products container
- Unindexed query on ProductsNoIndex: full scan, high RU expected

- High RU consumption: 880 RU/min — 8.8x above healthy baseline
- Azure Cosmos DB server-side latency: 18ms — within normal range, indicating service is healthy

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 18ms is healthy
- Azure SQL Database issue ruled out: SQL query latency 50ms, error count 0
- Host resource exhaustion ruled out: CPU at 26%
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Restore indexing policy on ProductsNoIndex, or route queries back to indexed Products container
- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review
- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review
- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT blame Azure SQL when SQL signals are healthy — SQL latency is 50ms
- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Keep escalation within the Contoso engineering response bridge while corrective actions are validated. Escalate externally only if ruled-out causes become active signals.

Root cause / permanent fix: Correct the Cosmos indexing and query design, then validate RU efficiency with representative load.

Engineering RCA: Telemetry shows p99 latency at 1250ms with 9 Cosmos 429 events, 880 RU/min consumption, host CPU 26%, and SQL latency 50ms. The active toggles (missingIndexing) directly align with the observed failure signature. The client-to-server latency delta of 1232ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

Multiple Cosmos clients (multiple-cosmos-clients)

SDK client created per request.

Primary classification	APPLICATION_OR_CLIENT_SIDE MATCH
Secondary classification(s)	—
Confidence	HIGH
Primary owner	Contoso Retail application engineering
Customer impact	Customers are experiencing degraded catalogue and checkout performance (p99 1450ms, checkout success 92%).

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 1450ms). Investigation evidence indicates the root cause is application-side, specifically: Client-side p99 latency: 1450ms vs server-side Cosmos latency: 14ms — 1436ms gap indicates client/SDK or host overhead; Cosmos DB 429 RateLimitExceeded: 14 in last 5 min on products container; Multiple CosmosClient instances: new client created per request instead of singleton. Azure Cosmos DB service-side health remained nominal (server-side latency: 14ms). Azure SQL Database remained healthy. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- Client-side p99 latency: 1450ms vs server-side Cosmos latency: 14ms — 1436ms gap indicates client/SDK or host overhead
- Cosmos DB 429 RateLimitExceeded: 14 in last 5 min on products container
- Multiple CosmosClient instances: new client created per request instead of singleton
- High RU consumption: 520 RU/min — 5.2x above healthy baseline
- Azure Cosmos DB server-side latency: 14ms — within normal range, indicating service is healthy

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 14ms is healthy
- Azure SQL Database issue ruled out: SQL query latency 55ms, error count 0
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Stop creating new CosmosClient per request — use singleton initialized at startup
- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review
- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review

- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT treat this as Azure/Cosmos DB service outage — server-side metrics are healthy
- DO NOT blindly increase provisioned RU/s — this masks the hot partition root cause
- DO NOT ignore hot partition evidence — increasing throughput without fixing key distribution does not resolve root cause
- DO NOT blame Azure SQL when SQL signals are healthy — SQL latency is 55ms
- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Do NOT escalate to Azure/Cosmos support. Evidence strongly indicates application-side patterns. Escalate only if server-side latency exceeds 200ms p99, availability drops below 99.9%, or Azure Service Health shows regional impact.

Root cause / permanent fix: Refactor Cosmos access patterns to use a singleton client, targeted point reads, well-distributed partition keys, and lean document shapes.

Engineering RCA: Telemetry shows p99 latency at 1450ms with 14 Cosmos 429 events, 520 RU/min consumption, host CPU 40%, and SQL latency 55ms. The active toggles (multipleClients) directly align with the observed failure signature. The client-to-server latency delta of 1436ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

Point read misuse (point-read-misuse)

Query used where point read should be used.

Primary classification	DATABASE_CONFIG_OR_DATA_MODEL MATCH
Secondary classification(s)	—
Confidence	HIGH
Primary owner	Contoso Retail data platform team
Customer impact	Customers are experiencing degraded catalogue and checkout performance (p99 1080ms, checkout success 98%).

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 1080ms). Investigation evidence indicates the root cause is most likely associated with the active failure domain, specifically: Client-side p99 latency: 1080ms vs server-side Cosmos latency: 17ms — 1063ms gap indicates client/SDK or host overhead; Cosmos DB 429 RateLimitExceeded: 6 in last 5 min on products container; Query used instead of ReadItem: SELECT * FROM c WHERE c.id=@id vs ReadItemAsync(id,pk). Azure Cosmos DB service-side health remained nominal (server-side latency: 17ms). Azure SQL Database remained healthy. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- Client-side p99 latency: 1080ms vs server-side Cosmos latency: 17ms — 1063ms gap indicates client/SDK or host overhead
- Cosmos DB 429 RateLimitExceeded: 6 in last 5 min on products container
- Query used instead of ReadItem: SELECT * FROM c WHERE c.id=@id vs ReadItemAsync(id,pk)
- High RU consumption: 610 RU/min — 6.1x above healthy baseline

- Azure Cosmos DB server-side latency: 17ms — within normal range, indicating service is healthy

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 17ms is healthy
- Azure SQL Database issue ruled out: SQL query latency 44ms, error count 0
- Host resource exhaustion ruled out: CPU at 23%
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Replace query with ReadItemAsync(id, partitionKey) for known id+pk combinations
- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review
- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review
- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT blame Azure SQL when SQL signals are healthy — SQL latency is 44ms
- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Keep escalation within the Contoso engineering response bridge while corrective actions are validated. Escalate externally only if ruled-out causes become active signals.

Root cause / permanent fix: Correct the Cosmos indexing and query design, then validate RU efficiency with representative load.

Engineering RCA: Telemetry shows p99 latency at 1080ms with 6 Cosmos 429 events, 610 RU/min consumption, host CPU 23%, and SQL latency 44ms. The active toggles (pointReadMisuse) directly align with the observed failure signature. The client-to-server latency delta of 1063ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

SQL connection pool pressure (sql-connection-pool-pressure)

Connections are held too long.

Primary classification	SQL_DATABASE_OR_SCHEMA MATCH
Secondary classification(s)	—
Confidence	HIGH
Primary owner	Contoso Retail SQL/data engineering team
Customer impact	Customers are experiencing degraded catalogue and checkout performance (p99 1650ms, checkout success 88%).

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 1650ms). Investigation evidence indicates the root cause is most likely associated with the active failure domain, specifically: Client-side p99 latency: 1650ms vs server-side Cosmos latency: 12ms — 1638ms gap indicates client/SDK or host overhead; SQL connection pool pressure: connections held beyond normal release; Azure Cosmos DB server-side latency: 12ms — within normal range, indicating service is healthy. Azure Cosmos DB service-side health remained nominal (server-side latency: 12ms). Azure SQL requires further validation. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- Client-side p99 latency: 1650ms vs server-side Cosmos latency: 12ms — 1638ms gap indicates client/SDK or host overhead
- SQL connection pool pressure: connections held beyond normal release
- Azure Cosmos DB server-side latency: 12ms — within normal range, indicating service is healthy

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 12ms is healthy
- Host resource exhaustion ruled out: CPU at 25%
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Disable SQL connection pressure — ensure connections returned to pool promptly
- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review
- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review
- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- DO NOT leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Escalate to the SQL/data engineering owner if latency remains above 1000ms after disabling synthetic pressure. Azure escalation is not required unless platform errors persist.

Root cause / permanent fix: Tune SQL schema and pool settings, then validate order-write latency under concurrency.

Engineering RCA: Telemetry shows p99 latency at 1650ms with 0 Cosmos 429 events, 65 RU/min consumption, host CPU 25%, and SQL latency 2100ms. The active toggles (sqlConnectionPressure) directly align with the observed failure signature. The client-to-server latency delta of 1638ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

SQL slow query (sql-slow-query)

Artificial SQL slowness injected.

Primary classification	SQL_DATABASE_OR_SCHEMA MATCH
Secondary classification(s)	—
Confidence	HIGH
Primary owner	Contoso Retail SQL/data engineering team
Customer impact	Customers are experiencing degraded catalogue and checkout performance (p99 1800ms, checkout success 90%).

Executive summary: During the incident window, Contoso Retail customers experienced degraded checkout and product catalogue performance (p99: 1800ms). Investigation evidence indicates the root cause is most likely associated with the active failure domain, specifically: Client-side p99 latency: 1800ms vs server-side Cosmos latency: 12ms — 1788ms gap indicates client/SDK or host overhead; SQL query latency: 3200ms — artificial slow query path enabled; Azure Cosmos DB server-side latency: 12ms — within normal range, indicating service is healthy. Azure Cosmos DB service-side health remained nominal (server-side latency: 12ms). Azure SQL requires further validation. Immediate corrective actions are identified and safe to execute without service escalation.

Evidence:

- Client-side p99 latency: 1800ms vs server-side Cosmos latency: 12ms — 1788ms gap indicates client/SDK or host overhead
- SQL query latency: 3200ms — artificial slow query path enabled
- Azure Cosmos DB server-side latency: 12ms — within normal range, indicating service is healthy

Ruled out:

- Azure Cosmos DB service-side outage ruled out: server-side p99 12ms is healthy
- Host resource exhaustion ruled out: CPU at 20%
- Azure regional outage ruled out: no Azure Service Health events detected

Immediate safe actions (no approval needed):

- Disable SQL slow query simulation — verify no N+1 query patterns in checkout path

- Verify singleton CosmosClient initialized at application startup and reused across all requests
- Verify SQL connection pool max size is appropriate for expected concurrency

Approval-required actions:

- Re-enable indexing policy on ProductsNoIndex container — requires change request, impacts write RU
- Scale AKS node pool — temporary mitigation pending root cause resolution and cost review
- Increase Cosmos DB throughput/autoscale — temporary mitigation only; pending cost/risk review
- Add SQL index — requires query plan review and DBA approval in production-like environment

DO NOT DO:

- **DO NOT** leave demo chaos toggles enabled — reset before any customer handoff

Escalation recommendation: Escalate to the SQL/data engineering owner if latency remains above 1000ms after disabling synthetic pressure. Azure escalation is not required unless platform errors persist.

Root cause / permanent fix: Tune SQL schema and pool settings, then validate order-write latency under concurrency.

Engineering RCA: Telemetry shows p99 latency at 1800ms with 0 Cosmos 429 events, 70 RU/min consumption, host CPU 20%, and SQL latency 3200ms. The active toggles (sqlSlowQuery) directly align with the observed failure signature. The client-to-server latency delta of 1788ms indicates most latency is introduced in the application, SDK, query design, or host execution path rather than inside the Azure Cosmos DB service plane.

5. Live End-to-End Walkthrough (real toggles, real traffic, not a fixture)

Instead of only replaying canned telemetry snapshots, a real incident was triggered against the running API (<http://localhost:3001>): `hotPartition`, `multipleClients`, and `highCpu` chaos toggles were enabled, 25 real product-browse requests were issued to generate genuine telemetry, and the SRE Agent investigated the live in-memory telemetry snapshot (not a fixture).

Primary classification	APPLICATION_OR_CLIENT_SIDE
Secondary classification(s)	HOST_OR_RUNTIME
Confidence	HIGH
Customer impact	Low-severity symptoms observed with p99 latency 297ms and checkout success 100%.

Evidence collected live:

- Cosmos DB 429 RateLimitExceeded: 25 in last 5 min on products container
- Hot partition pattern active: all writes routing to fixed partition key 'hot-category'
- Multiple CosmosClient instances: new client created per request instead of singleton
- Host CPU: 75% — CPU burn path active (fibonacci loop on request path)
- High RU consumption: 890 RU/min — 8.9x above healthy baseline

Safe action executed: `POST /api/chaos/reset` — disabled all chaos toggles (an "immediate safe action, no approval needed" per the agent's own recommendation).

Recovery verification (9 objective criteria)

Criterion	Result
p99 end-to-end latency returns to <200ms baseline	PENDING
Zero Cosmos DB 429 errors in 5-minute validation window	PENDING
Host CPU returns to <30% baseline	PASS
CosmosClient singleton verified — one instance at application startup	PASS
Query RU per operation <50 RU/s	PENDING
Unindexed query RU improves after indexing policy restored	PASS
Checkout order write succeeds in Azure SQL in <200ms	PASS

SQL query latency returns to <100ms	PASS
SQL connection pool utilization <60%	PASS

Why not all 9 passed immediately: host CPU, SQL, and singleton-client checks passed the instant the toggles were reset. The Cosmos 429 count check intentionally uses a rolling 5-minute evidence window — it does not zero out the instant you flip a switch. This is by design: the agent requires the evidence window to roll forward before declaring "recovered", the same discipline a real SRE would apply rather than trusting a single instantaneous reading. Overall recovered status: **PENDING (evidence window rolling forward)**.

6. ODE Reduction Outcome

Across the 14 scenarios exercised, **13 were classified as NOT requiring Azure support escalation** (application, data-model, host, or SQL-side root causes), and only the `azure-service-side-issue` scenario recommended considering an Azure Support case. In a real incident, this evidence-first classification step is what prevents unnecessary escalations, wasted engineer time, and support-case backlog — the core "ODE reduction" value proposition of Azure SRE Agent.

7. Environment Details

Repository	pronmalw/azure-sre-agent-ode-demo-kit (dev branch)
API	Node.js/Express, http://localhost:3001, in-memory fallback (no live Cosmos/SQL configured)
Web	React/Vite, http://localhost:3000
Classifier	src/api/src/services/sre-classifier.service.ts
Chaos toggles	hotPartition, metadataThrottling, multipleClients, highCpu, crossPartitionQuery, largeDocument, missingIndexing, pointReadMisuse, sqlSlowQuery, sqlConnectionPressure